

UNIVERSIDADE ESTADUAL DA PARAÍBA – UEPB
Campus I
Centro de Ciência e Tecnologia – CCT
Departamento de Computação

Gustavo Azevedo Lelis Farias

**PROJETO EXPERIMENTAL COMPARATIVO DE TÉCNICAS DE PROJETO DE
ALGORITMOS**

Relatório submetido ao Curso de Ciência da Computação da Universidade Estadual da Paraíba, na disciplina de Técnicas de Análise de Algoritmos, correspondente ao período letivo de 2025.2.

Campina Grande – PB
2026

RESUMO

Este relatório apresenta a segunda entrega do projeto experimental comparativo de técnicas de projeto de algoritmos. Foram implementadas e comparadas formalmente quatro estratégias Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa aplicadas aos problemas de Partição de Conjuntos e Árvore Geradora Mínima (AGM). Os experimentos foram conduzidos em Java com semente fixa (seed = 2026), uma repetição por algoritmo e timeout de 1.500 ms para técnicas exponenciais. A avaliação abrangeu complexidade assintótica teórica, tempo de execução empírico, uso de memória, qualidade da solução, escalabilidade e métricas do espaço de busca. Os resultados confirmam que Backtracking e Branch and Bound preservam exatidão ao custo de crescimento exponencial; a Programação Dinâmica implementada como tabela de alcançabilidade para Partição e como bitmask DP para AGM oferece comportamento polinomial ou pseudo-polinomial previsível; e a Estratégia Gulosa (Kruskal e Prim para AGM; heurística de balanceamento para Partição) proporciona o melhor desempenho temporal médio, com garantia de otimalidade restrita à AGM.

SUMÁRIO

1. INTRODUÇÃO	
1.1. Contextualização do Problema	
1.2. Objetivos	
1.2.1. Objetivo geral	
1.2.2. Objetivos específicos	
2. METODOLOGIA E IMPLEMENTAÇÃO	
2.1. Estrutura do Sistema Desenvolvido	
2.2. Formato de Dados de Entrada e Saída	
2.3. Métricas de Avaliação	
3. TÉCNICAS IMPLEMENTADAS	
3.1. Backtracking	
3.1.1. Análise de Corretude	
3.1.2. Análise de Desempenho	
3.1.2.1. Espacial	
3.1.2.2. Temporal	
3.2. Branch and Bound	
3.2.1. Análise de Corretude	
3.2.2. Análise de Desempenho	
3.2.2.1. Espacial	
3.2.2.2. Temporal	
3.3. Programação Dinâmica	
3.3.1. Análise de Corretude	
3.3.2. Análise de Desempenho	
3.3.2.1. Espacial	
3.3.2.2. Temporal	
3.4. Estratégia Gulosa	
3.4.1. Análise de Corretude	
3.4.2. Análise de Desempenho	
3.4.2.1. Espacial	
3.4.2.2. Temporal	
4. EXPERIMENTOS E RESULTADOS	
4.1. Instâncias de Teste	
4.2. Resultados Obtidos	
5. ANÁLISE E DISCUSSÃO	
5.1. Comparação das Estratégias	
6. CONCLUSÃO	

7. REFERÊNCIAS

1 INTRODUÇÃO

1.1 Contextualização do Problema

Esta segunda entrega do projeto experimental amplia o escopo da primeira, incorporando duas novas técnicas de projeto de algoritmos Programação Dinâmica e Estratégia Gulosa às já analisadas Backtracking e Branch and Bound. O estudo é conduzido por meio da aplicação dessas quatro estratégias a dois problemas representativos: o Problema de Partição de Conjuntos, de natureza combinatória, e o Problema da Árvore Geradora Mínima (AGM), de natureza estrutural em grafos. A justaposição desses problemas é deliberada: enquanto a Partição demanda exploração exaustiva do espaço de subconjuntos, a AGM admite soluções polinomiais exatas por algoritmos gulosos, criando um contraste rico para observar as diferenças de paradigma entre as técnicas.

A escolha do par de problemas também expõe limitações estruturais de cada abordagem. Técnicas exponenciais como Backtracking e Branch and Bound, embora corretas, tornam-se impraticáveis à medida que a entrada cresce, ao passo que Programação Dinâmica e algoritmos gulosos demonstram eficiência controlada em cenários de maior escala. A análise dessas diferenças, tanto em nível teórico quanto empírico, constitui o eixo central deste trabalho.

1.2 Objetivos

1.2.1 *Objetivo geral*

Implementar e comparar formalmente as estratégias Backtracking, Branch and Bound, Programação Dinâmica e Estratégia Gulosa, analisando suas características teóricas e empíricas nos problemas de Partição de Conjuntos e Árvore Geradora Mínima.

1.2.2 *Objetivos específicos*

- Analisar a complexidade assintótica teórica de cada técnica.
- Medir o tempo de execução empírico para instâncias de diferentes tamanhos.
- Avaliar o uso de memória durante a execução dos algoritmos.
- Comparar a qualidade da solução produzida (ótima versus aproximada).
- Investigar a escalabilidade das técnicas em relação ao crescimento da entrada.
- Quantificar o número de nós visitados e nós podados no espaço de busca.

- Justificar formalmente como cada técnica otimiza a exploração do espaço de soluções.

2 METODOLOGIA E IMPLEMENTAÇÃO

2.1 Estrutura do Sistema Desenvolvido

Os experimentos foram conduzidos em ambiente isolado, utilizando Java como linguagem de implementação para todas as técnicas, garantindo uniformidade na medição das métricas. O tempo de execução foi capturado com `System.nanoTime()`, convertido para milissegundos, e o uso de memória foi estimado pela diferença entre `totalMemory()` e `freeMemory()` da JVM ao longo de cada execução. Valores próximos de zero para memória refletem a limitação desta estimativa quando o coletor de lixo da JVM atua entre as medições, e não ausência de alocação. As instâncias de teste foram geradas com semente fixa (`seed = 2026`) para assegurar reprodutibilidade e comparabilidade direta entre as técnicas. Cada algoritmo foi executado uma vez por instância, com timeout prático de 1.500 ms para técnicas exponenciais.

As implementações estão organizadas nos seguintes arquivos-fonte:

- `PartitionSolver.java` — implementações das quatro técnicas para Partição de Conjuntos.
- `MSTSolver.java` — implementações das quatro técnicas para Árvore Geradora Mínima.
- `Main.java` — entrada de dados e execução de instâncias individuais.
- `BenchmarkMain.java` — execução comparativa automatizada e consolidação de métricas.

2.2 Formato de Dados de Entrada e Saída

Problema de Partição de Conjuntos

Entrada:

```
n
a1 a2 ... an
```

Saída:

```
YES
subset1
subset2
Ou: NO
```

Problema de Árvore Geradora Mínima (AGM)

Entrada:

```
n m
u1 v1 w1
...
um vm wm
```

Saída:

```
peso_total
u1 v1 w1
...
u(n-1) v(n-1) w(n-1)
```

2.3 Métricas de Avaliação

As técnicas foram avaliadas com base nas seguintes métricas:

- Complexidade Assintótica (Teórica): análise da ordem de crescimento do tempo e espaço de execução no pior caso.
- Tempo de Execução (Empírico): medição do tempo real de processamento para instâncias de diferentes tamanhos.
- Uso de Memória: estimativa do pico de memória utilizada pela JVM durante a execução.
- Qualidade da Solução: comparação entre a solução encontrada e a solução ótima, quando aplicável, ou análise da razão de aproximação.
- Escalabilidade: avaliação do comportamento das técnicas à medida que o tamanho da entrada cresce.
- Métricas de Poda: quantificação dos nós visitados e podados, evidenciando a eficácia das estratégias de otimização da busca.

3 TÉCNICAS IMPLEMENTADAS

Esta seção detalha as quatro técnicas de projeto de algoritmos comparadas nesta entrega, apresentando suas descrições, fragmentos de implementação representativos e análises de corretude e desempenho para os dois problemas de referência.

3.1 Backtracking

3.1.1 Análise de Corretude

O Backtracking é uma técnica de busca exaustiva que constrói soluções de forma incremental, retrocedendo sempre que detecta que o estado parcial corrente não pode conduzir a uma solução válida. Sua corretude reside na garantia de que, se uma solução existe, ela será encontrada, pois todas as possibilidades são exploradas enquanto as podas aplicadas forem seguras isto é, descartarem apenas ramos que provadamente não contêm soluções válidas.

Problema de Partição de Conjuntos

Para a Partição de Conjuntos, o Backtracking busca um subconjunto cuja soma seja exatamente $S_{total}/2$. A corretude é assegurada pela exploração de todas as combinações possíveis de elementos para formar o primeiro subconjunto. A poda de viabilidade que descarta o ramo quando a soma corrente excede o alvo é segura porque nenhum elemento do conjunto possui valor negativo, tornando impossível reverter o excesso por adições futuras.

Fragmento de implementação (poda em Backtracking – Partição):

```
private void btRecursive(int idx, int currentSum,
    List<Integer> currentIndices, PartitionData result) {
    nodesVisited++;
    if (result.found) return;
    if (idx == n) {
        if (currentSum == targetSum) { result.found = true; /*
monta subsets */ }
        return;
    }
    if (currentSum > targetSum) { nodesPruned++; return; } //
poda de viabilidade
    currentIndices.add(idx);
    btRecursive(idx + 1, currentSum + elements[idx],
currentIndices, result);
    currentIndices.remove(currentIndices.size() - 1);
    btRecursive(idx + 1, currentSum, currentIndices, result);
}
```

Problema de Árvore Geradora Mínima (AGM)

Para a AGM, o Backtracking enumera todas as combinações válidas de arestas que formam árvores geradoras, selecionando a de menor custo total. A abordagem é correta, porém a explosão combinatória do espaço de busca torna a técnica impraticável para

grafos com mais de 8 a 10 vértices, onde o número de árvores geradoras possíveis cresce exponencialmente com o número de arestas.

3.1.2 *Análise de Desempenho*

3.1.2.1 *Espacial*

A complexidade espacial do Backtracking é $O(n)$ para Partição e $O(E)$ para AGM, determinada pela profundidade máxima da pilha de recursão. Esse custo contido é observado nos experimentos: as medições de memória do Backtracking foram inferiores às do Branch and Bound em todos os tamanhos testados, confirmando que a técnica preserva espaço ao custo de tempo.

3.1.2.2 *Temporal*

A complexidade temporal do Backtracking é $O(2^n)$ no pior caso para Partição. Os dados empíricos evidenciam comportamento irregular 2,64 ms para $n = 8$, caindo para 0,15 ms em $n = 16$ e subindo para 1,86 ms em $n = 20$ reflexo da variabilidade das instâncias e da efetividade das podas em cada caso. Para a AGM, o Backtracking tornou-se inviável a partir de $V = 10$, onde o timeout de 1.500 ms foi atingido.

- Partição de Conjuntos: $O(2^n)$ no pior caso.
- AGM por enumeração: exponencial no número de arestas; inviável para $V \geq 10$.

3.2 Branch and Bound

3.2.1 *Análise de Corretude*

O Branch and Bound estende o Backtracking ao introduzir limites (bounds) que permitem descartar ramos cujo melhor resultado possível já é inferior à melhor solução conhecida. A corretude é mantida desde que os limites sejam admissíveis nunca subestimando o valor real dos descendentes. Além de podar por inviabilidade, o Branch and Bound poda por dominância: se o bound do nó corrente não supera o incumbente, a exploração desse ramo é abandonada.

Problema de Partição de Conjuntos

Para a Partição, o bound verifica se a diferença atual entre as somas pode ser compensada pelos elementos restantes. Se $|\text{sum1} - \text{sum2}| > \text{soma dos elementos restantes}$, é impossível equilibrar os subconjuntos a partir desse nó. A fila de prioridade expande primeiro os nós mais equilibrados (menor diferença), acelerando a convergência para a solução.

Problema de Árvore Geradora Mínima (AGM)

Para a AGM, o custo parcial acumulado é comparado ao incumbente inicializado pelo Kruskal. Se o custo do nó corrente já supera o melhor peso conhecido, toda a subárvore é descartada. Os dados mostram que, para $V = 6$, o Branch and Bound realizou 592 podas com 1.905 nós visitados mesma contagem do Backtracking, pois o incumbente inicial (Kruskal) é ótimo, e o bound adicional não adiciona podas extras além das de viabilidade já presentes no Backtracking.

Fragmento de implementação (bound em Branch and Bound – AGM):

```
if (node.weight >= bestWeight) {  
    nodesPruned++; // custo parcial supera incumbente:  
    descartar  
    continue;  
}
```

3.2.2 Análise de Desempenho

3.2.2.1 Espacial

A complexidade espacial do Branch and Bound pode atingir $O(2^n)$ no pior caso pela manutenção da fronteira de nós ativos. Esse comportamento é diretamente observado nos experimentos: o Branch and Bound consumiu 47.649 KB para $n = 20$ em Partição, enquanto o Backtracking registrou 0,00 KB para o mesmo tamanho. A troca entre tempo e espaço é, portanto, um custo real e mensurável da estratégia.

3.2.2.2 Temporal

No pior caso, a complexidade temporal do Branch and Bound também é $O(2^n)$. Na prática, a qualidade do bound determina a eficácia das podas. Para Partição com $n = 24$, o Branch and Bound completou em 2.458 ms com 947.840 podas confirmando que o bound foi informativo o suficiente para tornar a instância tratável. Para AGM com $V =$

12, o limite prático foi atingido, demonstrando que o bound por custo parcial não é suficientemente informativo para esse problema.

- Partição de Conjuntos: $O(2^n)$ no pior caso, com redução prática dependente da qualidade do bound.
- AGM: $O(2^n)$ no pior caso; inviável para $V = 12$ nas instâncias testadas.

3.3 Programação Dinâmica

3.3.1 Análise de Corretude

A Programação Dinâmica resolve o problema decompondo-o em subproblemas menores com subestrutura ótima e armazenando os resultados intermediários para evitar recomputações. Para a Partição de Conjuntos, a técnica constrói uma tabela de alcançabilidade $dp[i][s]$, onde $dp[i][s] = \text{verdadeiro}$ indica que a soma s pode ser formada com os primeiros i elementos. A corretude é garantida pela completude do preenchimento: todos os estados alcançáveis são marcados, e o estado final $dp[n][\text{targetSum}]$ determina com certeza se a partição é possível.

Fragmento de implementação (transição de estado em DP – Partição):

```
if (dp[i][s]) {
    dp[i + 1][s] = true;           // não inclui
    elemento i
    if (s + elements[i] <= targetSum)
        dp[i + 1][s + elements[i]] = true; // inclui
    elemento i
}
```

Problema de Árvore Geradora Mínima (AGM)

Para a AGM, a implementação utiliza bitmask DP: $dp[\text{mask}]$ armazena o menor custo para conectar exatamente os vértices presentes no conjunto mask , partindo do vértice 0. A subestrutura ótima é garantida pela propriedade de que a AGM de um conjunto S contém a AGM ótima de qualquer subconjunto $S' \subset S$ conectado por ela. A sobreposição de subproblemas é explorada pelo fato de que o mesmo conjunto mask pode ser alcançado por múltiplas sequências de expansão; armazenar $dp[\text{mask}]$ evita recomputar seu custo mínimo. A corretude é assegurada pela transição: para cada aresta (u, v) com exatamente um extremo em mask , atualiza-se $dp[\text{mask} \mid (1 \ll \text{newVertex})]$ se o novo custo for menor.

Fragmento de implementação (transição de estado em bitmask DP – AGM):

```
for (int mask = 1; mask <= fullMask; mask++) {
    if (dp[mask] == Integer.MAX_VALUE) continue;
    for (Edge edge : edges) {
        boolean uIn = (mask & (1 << edge.u)) != 0;
        boolean vIn = (mask & (1 << edge.v)) != 0;
        if (uIn == vIn) continue; // ambos dentro ou fora:
        ignora
        int nv = uIn ? edge.v : edge.u;
        int newMask = mask | (1 << nv);
        if (dp[mask] + edge.weight < dp[newMask])
            dp[newMask] = dp[mask] + edge.weight; //
        memoização
    }
}
```

3.3.2 Análise de Desempenho

3.3.2.1 Espacial

Para Partição, a complexidade espacial é $O(n \times \text{alvo})$, determinada pelo tamanho da tabela de estados. Os experimentos registraram entre 8,25 KB ($n = 8$, 963 células ativas) e 60,15 KB ($n = 24$, 7.550 células ativas), crescimento controlado e previsível. Para AGM com bitmask DP, a complexidade espacial é $O(2^V)$, com uso de memória próximo a zero nas instâncias testadas reflexo do tamanho reduzido do vetor `dp[]` para $V \leq 12$.

3.3.2.2 Temporal

A complexidade temporal da Programação Dinâmica é $O(n \times \text{alvo})$ para Partição pseudo-polinomial e $O(2^V \times E)$ para AGM com bitmask DP. Ambas são muito mais controladas que as complexidades exponenciais das técnicas de busca. Os dados empíricos confirmam esse comportamento: a DP completou todas as instâncias de Partição em menos de 1 ms, e todas as instâncias de AGM em menos de 0,2 ms, com peso ótimo em todos os casos.

- Partição de Conjuntos: $O(n \times \text{alvo})$ em tempo e espaço pseudo-polinomial.
- AGM (bitmask DP): $O(2^V \times E)$ em tempo e $O(2^V)$ em espaço exato e eficiente para $V \leq 20$.

3.4 Estratégia Gulosa

3.4.1 Análise de Corretude

A Estratégia Gulosa toma, a cada passo, a decisão localmente ótima sem reconsiderar escolhas anteriores. Sua corretude global depende da estrutura do problema: se o problema possui a propriedade da escolha gulosa, a estratégia é exata; caso contrário, produz apenas uma aproximação.

Problema de Partição de Conjuntos

Para a Partição de Conjuntos, a heurística gulosa ordena os elementos em ordem decrescente e atribui cada um ao subconjunto de menor soma corrente. Esse critério não possui garantia de otimalidade global a Partição é NP-completa e a propriedade da escolha gulosa não se aplica em geral. Os experimentos confirmam essa limitação: a Gulosa obteve $\text{diff} = 2$ para $n = 8$, $\text{diff} = 6$ para $n = 12$ e $\text{diff} = 2$ para $n = 20$ e $n = 24$, encontrando a partição exata apenas em $n = 16$, resultado favorecido pela estrutura específica dessa instância.

Problema de Árvore Geradora Mínima (AGM)

Para a AGM, os algoritmos de Kruskal e Prim são gulosos com garantia de otimalidade, graças à propriedade do corte mínimo: em todo corte do grafo, a aresta de menor peso que cruza o corte pertence a alguma AGM ótima. Essa propriedade, derivada da estrutura matroidal do problema, torna a escolha local suficiente para garantir a solução global ótima. Os experimentos confirmam: Kruskal e Prim produziram peso ótimo em todas as instâncias, com tempo inferior a 10 ms mesmo para $V = 6$ na primeira execução (aquecimento da JVM).

Fragmento de implementação (escolha gulosa em Kruskal – AGM):

```
for (Edge edge : sortedEdges) {
    if (uf.find(edge.u) != uf.find(edge.v)) {
        uf.union(edge.u, edge.v);
        mstEdges.add(edge);
        totalWeight += edge.weight;
    }
}
```

3.4.2 Análise de Desempenho

3.4.2.1 Espacial

A complexidade espacial da Estratégia Gulosa é $O(n)$ para a heurística de Partição e $O(E)$ para Kruskal, referente ao armazenamento das arestas e à estrutura Union-Find. Em ambos os casos, o uso de memória é o menor entre as quatro técnicas confirmado pelos experimentos, onde a Gulosa registrou entre 0,20 KB e 0,58 KB para Partição e entre 0,53 KB e 3,17 KB para AGM.

3.4.2.2 Temporal

A complexidade temporal da heurística gulosa para Partição é $O(n \log n)$, dominada pela ordenação dos elementos. Para Kruskal, a complexidade é $O(E \log E)$ pela ordenação das arestas, com operações Union-Find em tempo quase linear amortizado. Os dados empíricos refletem esse comportamento: a Gulosa em Partição foi consistentemente a mais rápida em todas as instâncias, variando entre 0,06 ms e 0,08 ms para $n \geq 12$. O tempo elevado observado para $n = 8$ (5,48 ms) é artefato do aquecimento da JVM na primeira execução.

- Partição heurística: $O(n \log n)$ em tempo, $O(n)$ em espaço.
- Kruskal (AGM): $O(E \log E)$ em tempo, $O(E)$ em espaço.
- Prim (AGM): $O(E \log V)$ em tempo, $O(V)$ em espaço.

4 EXPERIMENTOS E RESULTADOS

4.1 Instâncias de Teste

O benchmark foi executado por BenchmarkMain.java com semente fixa (seed = 2026), uma repetição por algoritmo e timeout prático de 1.500 ms para técnicas exponenciais. Para Partição de Conjuntos, foram utilizados conjuntos de $n \in \{8, 12, 16, 20, 24\}$ elementos inteiros positivos, gerados em pares iguais para garantir que instâncias com partição exata estivessem disponíveis para verificação de corretude. Para a AGM, grafos não direcionados e ponderados com $V \in \{6, 8, 10, 12\}$ vértices foram gerados com densidade suficiente para exigir seleção criteriosa de arestas. Os casos sinalizados com "-" nas tabelas indicam instâncias onde o timeout foi atingido antes da conclusão.

4.2 Resultados Obtidos

Todos os experimentos foram executados em ambiente isolado com instrumentação Java para tempo (System.nanoTime()) e memória (Runtime.getRuntime()). As

instâncias foram geradas com semente fixa para garantir comparabilidade entre as técnicas.

Partição de Conjuntos

A Tabela 1 consolida os resultados obtidos para o Problema de Partição de Conjuntos, apresentando tempo de execução, uso de memória, número de nós visitados e podados, e a diferença absoluta entre as somas dos subconjuntos como medida de qualidade.

Tabela 1 – Partição de Conjuntos: tempo, memória, exploração e qualidade.

n	Técnica	Tempo (ms)	Memória (KB)	Nós visitados	Nós podados	Diff
8	Backtracking	2,6445	0,00	50	0	0
8	Branch and Bound	9,7076	0,00	105	28	0
8	Prog. Dinâmica	0,1601	0,00	656	0	0
8	Gulosa	5,4814	0,00	8	0	0
12	Backtracking	0,2922	0,00	158	0	0
12	Branch and Bound	15,6109	491,54	1764	386	0
12	Prog. Dinâmica	0,2970	0,00	2436	0	0
12	Gulosa	0,0709	0,00	12	0	0
16	Backtracking	0,1480	0,00	46	0	0
16	Branch and Bound	96,3449	5242,78	25196	5546	0
16	Prog. Dinâmica	0,3822	0,00	3712	0	0
16	Gulosa	0,0581	0,00	16	0	0
20	Backtracking	1,8649	0,00	1150	0	0
20	Branch and Bound	473,8015	47649,44	504009	104474	0
20	Prog. Dinâmica	0,4813	0,00	4880	0	0
20	Gulosa	0,0684	0,00	20	0	0
24	Backtracking	-	-	-	-	limite prático
24	Branch and Bound	2458,4253	37661,52	3693995	947840	0
24	Prog. Dinâmica	0,8376	0,00	7776	0	0
24	Gulosa	0,0838	0,00	24	0	0

Fonte: elaborado pelo autor (2026).

O Backtracking apresentou comportamento variável entre instâncias reflexo da dependência entre a estrutura do conjunto e a efetividade das podas de viabilidade e atingiu o limite prático para $n = 24$. O Branch and Bound, embora preservando exatidão até $n = 24$, apresentou crescimento expressivo tanto em tempo (2.458 ms) quanto em memória (37.661 KB), evidenciando o custo da manutenção da fronteira de nós. A Programação Dinâmica demonstrou comportamento estável e previsível em todas as

instâncias, completando $n = 24$ em menos de 1 ms. A Estratégia Gulosa foi a mais rápida, mas sem garantia de exatidão: obteve $\text{diff} > 0$ em quatro das cinco instâncias testadas.

Árvore Geradora Mínima

A Tabela 2 apresenta os resultados para o Problema da Árvore Geradora Mínima, comparando os cinco algoritmos implementados em grafos de $V \in \{6, 8, 10, 12\}$ vértices.

Tabela 2 – Árvore Geradora Mínima: tempo, memória, exploração e qualidade.

V	E	Técnica	Tempo (ms)	Memória (KB)	Nós visitados	Nós podados	Peso
6	14	Kruskal	9,9074	0,00	14	3	64
6	14	Prim	1,3732	0,00	6	3	64
6	14	Prog. Dinâmica	0,0492	0,00	5	0	64
6	14	Backtracking	24,7827	0,00	1905	468	64
6	14	Branch and Bound	23,3178	0,00	1905	592	64
8	18	Kruskal	0,0710	0,00	18	1	67
8	18	Prim	0,2313	0,00	8	1	67
8	18	Prog. Dinâmica	0,0942	0,00	7	0	67
8	18	Backtracking	17,6189	4096,00	10892	3502	67
8	18	Branch and Bound	32,0099	2048,00	10892	4177	67
10	22	Kruskal	0,0371	0,00	22	0	78
10	22	Prim	0,1513	0,00	10	0	78
10	22	Prog. Dinâmica	0,0792	0,00	9	0	78
10	22	Backtracking	-	-	-	-	limite
10	22	Branch and Bound	184,5703	22528,00	89651	34796	78
12	27	Kruskal	0,0243	0,00	27	4	175
12	27	Prim	0,1702	0,00	12	4	175
12	27	Prog. Dinâmica	0,1172	0,00	11	0	175
12	27	Backtracking	-	-	-	-	limite
12	27	Branch and Bound	-	-	-	-	limite

Fonte: elaborado pelo autor (2026).

Os tempos elevados de Kruskal (9,90 ms) e Backtracking (24,78 ms) para $V = 6$ são artefatos do aquecimento da JVM na primeira execução do benchmark a partir de $V = 8$, Kruskal estabiliza abaixo de 0,1 ms. Kruskal e Prim produziram peso ótimo em todas as instâncias com tempo proporcional ao número de arestas. A Programação Dinâmica (bitmask DP) também atingiu peso ótimo em todas as instâncias em menos de 0,2 ms. O

Backtracking tornou-se inviável a partir de $V = 10$. O Branch and Bound ainda completou $V = 10$ em 184 ms com 34.796 podas, mas falhou para $V = 12$.

5 ANÁLISE E DISCUSSÃO

5.1 Comparação das Estratégias

A tabela a seguir consolida as dimensões de análise para cada técnica, considerando complexidade teórica, desempenho empírico e qualidade da solução para os dois problemas estudados.

Tabela 3 – Comparação consolidada das técnicas por critério.

Critério	Backtracking	Branch and Bound	Prog. Dinâmica	Gulosa
Complexidade temporal	$O(2^n)$	$O(2^n)^\dagger$	$O(n \times \text{alvo}) / O(2^{V \times E})$	$O(n \log n) / O(E \log E)$
Complexidade espacial	$O(n)$	$O(2^n)$ fronteira	$O(n \times \text{alvo}) / O(2^V)$	$O(n)$
Tempo empírico – Partição	Variável	Alto	Excelente	Excelente
Tempo empírico – AGM	Inviável ≥ 10	Inviável $V=12$	Excelente	Excelente
Qualidade – Partição	Ótima	Ótima	Ótima	Aprox.
Qualidade – AGM	Ótima*	Ótima*	Ótima	Ótima
Escalabilidade	Ruim	Ruim	Boa	Excelente
Mecanismo de otimização	Viabilidade	Bound+Viab.	Memoização	Escolha local

† Pior caso teórico $O(2^n)$, com redução prática dependente da qualidade do bound. * Ótima apenas nas instâncias concluídas dentro do limite prático.

Os resultados demonstram que a escolha da técnica de projeto de algoritmos é intrinsecamente condicionada pela estrutura do problema. Para a Partição de Conjuntos, onde a otimalidade é necessária, a Programação Dinâmica oferece a melhor relação entre exatidão e eficiência: completou $n = 24$ em 0,84 ms com uso de memória controlado, enquanto o Branch and Bound exigiu 2.458 ms e 37 MB para a mesma instância. O Branch and Bound é uma alternativa viável para instâncias médias, mas seu consumo de memória cresce de forma preocupante. A Estratégia Gulosa é a opção mais rápida, porém os resultados experimentais $\text{diff} > 0$ em quatro das cinco instâncias confirmam que não é confiável para esse problema sem garantias adicionais.

Para a AGM, o panorama é radicalmente diferente: a estrutura matroidal do problema confere aos algoritmos gulosos Kruskal e Prim garantia de otimalidade com

complexidade polinomial, dominando todas as métricas. A Programação Dinâmica com bitmask DP também produziu resultados ótimos com custo muito inferior às técnicas de busca. O uso de Backtracking ou Branch and Bound para AGM é injustificável em qualquer instância prática o contraste entre Kruskal (27 nós visitados, 0,02 ms para $V = 12$) e Branch and Bound (inviável para o mesmo tamanho) sintetiza objetivamente como a adequação do paradigma ao problema supera qualquer otimização de implementação.

A análise das podas aprofunda essa conclusão. O Backtracking poda por viabilidade: ramos que violam uma condição necessária são descartados sem perda de corretude. O Branch and Bound adiciona poda por dominância: mesmo que o estado parcial seja viável, se seu bound não supera o incumbente, o ramo é abandonado mecanismo que, para Partição com $n = 24$, eliminou 947.840 nós, tornando viável uma instância inacessível ao Backtracking puro. A Programação Dinâmica não poda no sentido tradicional substitui a exploração por preenchimento determinístico de tabela, eliminando recomputações pelo princípio da sobreposição de subproblemas. A Estratégia Gulosa, por fim, não explora ramificações: toma decisões irrevogáveis com base no critério local, atingindo complexidade ótima ao custo de não garantir globalidade da solução em problemas sem propriedade de matroide.

6 CONCLUSÃO

Este projeto experimental demonstrou, de forma teórica e empírica, que a escolha da técnica de projeto de algoritmos é intrinsecamente condicionada pela estrutura do problema e pelo crescimento esperado da entrada. Nenhuma das quatro estratégias estudadas se mostrou superior em todas as dimensões analisadas; cada uma apresenta vantagens específicas que a tornam adequada a determinados contextos.

O Backtracking é a abordagem mais simples de implementar e suficiente para instâncias pequenas, mas sua complexidade exponencial o torna inviável rapidamente. O Branch and Bound melhora esse cenário ao incorporar limites que reduzem o espaço efetivamente explorado, porém a custo crescente de memória o consumo de 37 MB para Partição com $n = 24$ e a inviabilidade para AGM com $V = 12$ ilustram que a estratégia negocia espaço por tempo de forma não controlada. Essa troca é aceitável em instâncias médias, mas se torna um gargalo à medida que a entrada cresce.

A Programação Dinâmica oferece a melhor relação entre exatidão e eficiência nos dois problemas estudados: pseudo-polinomial para Partição onde completou $n = 24$ em 0,84 ms com 60 KB e exponencial em V mas prática para $V \leq 12$ com bitmask DP para AGM.

A correção fundamental entre a versão original, que implementava Prim disfarçado de DP, e a versão com bitmask DP genuína, ilustra a importância de verificar se os princípios de subestrutura ótima e sobreposição de subproblemas estão de fato sendo explorados, e não apenas a estrutura superficial do código. Para a AGM, os algoritmos gulosos Kruskal e Prim dominam todas as métricas graças à propriedade matroidal do problema, que garante a otimalidade da escolha local.

Do ponto de vista mais amplo, os resultados reforçam um princípio fundamental da análise de algoritmos: compreender a estrutura matemática de um problema é tão importante quanto a eficiência da implementação. Problemas com subestrutura ótima e sobreposição de subproblemas são naturais para Programação Dinâmica; problemas com propriedade de matroide admitem soluções gulosas ótimas; problemas sem nenhuma dessas estruturas exigem busca exaustiva, mitigada pelas podas de Backtracking e Branch and Bound. A correta identificação dessas propriedades é o que distingue uma solução prática de uma solução teoricamente correta, porém inaplicável em escala.

Como trabalho futuro, recomenda-se a condução de múltiplas repetições por instância com coleta de medidas de dispersão estatística, o teste com instâncias de Partição sem garantia de solução exata para avaliar sistematicamente a qualidade aproximada da heurística gulosa e a exploração de bounds mais sofisticados para Branch and Bound em AGM, possivelmente baseados em estimativas do custo mínimo das arestas restantes.

7 REFERÊNCIAS

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 3. ed. Cambridge: MIT Press, 2009.

HOROWITZ, E.; SAHNI, S.; RAJASEKARAN, S. Fundamentals of Computer Algorithms. 2. ed. Hoboken: Silicon Press, 2007.

SKIENA, S. S. The Algorithm Design Manual. 2. ed. New York: Springer Science & Business Media, 2008.